

Typing Rules and Evaluation Rules in Block-MiniJava

A Formal Reconstruction of the Implemented Static and Dynamic Semantics

Research note — Block-MiniJava

2026-07-20

Abstract

Block-MiniJava’s inspector exposes two kinds of formal artifact at runtime: a *Typing* tab that renders on-demand natural-deduction derivations for every method, and three small-step *steppers* (CESK, A vs B, Rewrite) that expose the program’s operational semantics one transition at a time. Both artifacts are produced by the same code that decides whether a program type-checks and what it evaluates to — the visualization cannot drift from the implementation because it *is* the implementation, instrumented to record its own derivation. This note reconstructs, rule by rule, the static type system (§2) and the two operational semantics — an abstract machine and a substitution-based reduction semantics (§3) — exactly as implemented in `src/core/types/typeChecker.ts`, `src/core/types/classTable.ts`, `src/core/semantics/minijavaMachine.ts`, and `src/core/semantics/minijavaSubstitution.ts`, and traces each rule family to the programming-language-theory literature it descends from.

Contents

1	Introduction	2
2	The Static Type System	2
2.1	Type grammar	2
2.2	Contexts, the class table, and subtyping	3
2.3	Assignability: subtyping plus hole consistency	3
2.4	Expression typing rules (T-*)	3
2.5	Statement well-formedness rules (WF-*)	5
2.6	Method well-formedness: M-OK	5
2.7	Method overriding	5
2.8	Theoretical sources	6
3	Operational Semantics	6
3.1	Vehicle 1: a store-bearing abstract machine, parameterized by value model	6
3.1.1	Configurations	6
3.1.2	Continuations	7
3.1.3	Transition rules	7
3.2	Vehicle 2: substitution-based reduction semantics (pure fragment)	10
3.2.1	Scope: the pure fragment	10
3.2.2	Values and evaluation contexts	10
3.2.3	Reduction rules	10
3.2.4	The β rule: method call	11
3.3	Operational correspondence between the two vehicles	11
3.4	An auxiliary evaluation rule system: mark-and-sweep garbage collection	11
3.5	Theoretical sources	12

1 Introduction

Block-MiniJava implements MiniJava (the CSE 401 core, plus a `String` extension; see `MiniJava-Adjusted-BNF.md`) as a block-based language with:

- a three-layer static type system (type grammar, class table with nominal subtyping, and a bidirectional checker) that also produces a typing *derivation* for the Typing tab, and
- two independently implemented, cross-checked operational semantics:
 1. **Vehicle 1** — a small-step *abstract machine*, parameterized by a *value model* $\in \{A, B\}$: Model A is faithful MiniJava (a mutable heap, reference semantics, aliasing); Model B is structure-preserving (objects/arrays are inline values, field update is functional). Both models share one control-flow skeleton, so they can be run in lockstep on the same program (the “A vs B” tab).
 2. **Vehicle 2** — a small-step *substitution* (reduction) semantics over the pure fragment of the language: every state is a literal block tree, and a step is a literal rewrite of a redex (the “Rewrite” tab). An *operational correspondence* check replays the same program on the Model B machine and asserts that the two traces agree on every *salient* rule.

Every rule below is named exactly as it appears in the source (the `rule` field of a `Derivation`, or the `lastRule` / `rule` field of a machine or substitution step), so this note can be read side by side with the Typing, CESK, A-vs-B, and Rewrite tabs.

2 The Static Type System

The type system is implemented in three separable layers, following the design document’s own layering (`block-minijava-value-and-visualisation-design.md`, §1):

1. **Type grammar** (`core/types/ty.ts`) — the abstract syntax of types.
2. **Class table** (`core/types/classTable.ts`) — name resolution, `extends` chains, nominal subtyping.
3. **Checker** (`core/types/typeChecker.ts`) — the typing and statement well-formedness judgments, which also emit a `Derivation` tree consumed by the Typing tab.

2.1 Type grammar

$$\tau ::= \text{int} \mid \text{boolean} \mid \text{String} \mid \text{int}[] \mid \text{String}[] \mid C \mid (\tau_1, \dots, \tau_n) \rightarrow \tau \mid \text{Object} \mid \text{Nothing} \mid ?$$

where C ranges over declared class names. `Object` (Top) and `Nothing` (Bottom) bound the nominal subtype lattice; the arrow type $(\tau_1, \dots, \tau_n) \rightarrow \tau$ is produced only internally, as the result of the mtype lookup used by T-INVK (§2.4) — there is no arrow-typed surface syntax. `?` is a *first-class unfilled-annotation* marker in the style of Hazel’s typed holes (Omar et al. [3, 4]): an empty socket, or an expression whose type could not be determined, types as `?` and is made consistent with every other type (§2.3) so that one incomplete block never cascades into unrelated errors.

2.2 Contexts, the class table, and subtyping

A typing context Γ is a finite map from identifiers to types. Inside a method of class C , Γ is fixed for the whole body:

$$\Gamma = \{ \text{this} : C \} \cup \{ x_i : \tau_i \mid x_i \text{ a parameter} \} \cup \{ y_j : \tau_j \mid y_j \text{ a local variable} \}$$

(**this** is omitted inside **main**, whose sole binding is **args**: **String**[]). Field lookup is a *separate*, class-indexed environment $\text{fields}(C)$, resolved by walking C and its ancestors and taking the nearest (most-derived) declaration of a name — an identifier that is not in Γ but is in $\text{fields}(C)$ is an *implicit* **this.f** reference. Method lookup is the analogous $\text{mtype}(m, C)$, giving the $(\tau_1, \dots, \tau_n) \rightarrow \tau$ signature found by the same nearest-ancestor walk (overriding, not overloading: MiniJava has one method per name per class). Both $\text{fields}(\cdot)$ and $\text{mtype}(\cdot, \cdot)$ are named after, and behave exactly as, the identically named auxiliary functions in Featherweight Java [2] (§2.8).

Subtyping $\tau <: \tau'$ is nominal, seeded from **extends**:

$$\begin{array}{c} \frac{}{\tau <: \tau} \text{ (S-REFL)} \quad \frac{}{\tau <: \text{Object}} \text{ (S-TOP)} \quad \frac{}{\text{Nothing} <: \tau} \text{ (S-BOTTOM)} \\[10pt] \frac{\text{class } C \text{ extends } D}{C <: D} \text{ (S-EXTENDS)} \quad \frac{C <: D \quad D <: E}{C <: E} \text{ (S-TRANS)} \end{array}$$

(the implementation gets S-TRANS for free: **isSubtype** walks the whole ancestor chain of C looking for D , rather than composing single **extends** steps.) Two classes with no subtype relationship except through **Object** are otherwise incomparable — there is no structural or union/intersection subtyping.

2.3 Assignability: subtyping plus hole consistency

Every place a value flows into a typed slot (assignment, argument passing, **return**) is checked against *assignability* $\tau \rightsquigarrow \tau'$, not raw subtyping:

$$\frac{\tau <: \tau'}{\tau \rightsquigarrow \tau'} \text{ (ASG-SUB)} \quad \frac{}{? \rightsquigarrow \tau'} \text{ (ASG-HOLEL)} \quad \frac{}{\tau \rightsquigarrow ?} \text{ (ASG-HOLER)}$$

This is exactly a two-sided *gradual-typing consistency* relation $\tau \sim \tau'$ in the sense of Siek and Taha [5], restricted so that the only “unknown” type is the editor’s typed hole — the same design Hazel uses to keep an editable, possibly-incomplete program meaningfully typed at every keystroke [3].

2.4 Expression typing rules (T-*)

Judgment: $\Gamma \vdash e : \tau$ — “under Γ , expression e has type τ .” Each rule below is named after the rule string the checker attaches to the corresponding **Derivation** node in **typeChecker.ts**.

Literals and variables.

$$\begin{array}{c} \frac{}{\Gamma \vdash n : \text{int}} \text{ (T-INT)} \quad \frac{}{\Gamma \vdash "s" : \text{String}} \text{ (T-STR)} \quad \frac{}{\Gamma \vdash b : \text{boolean}} \text{ (T-BOOL)} \quad (b \in \{\text{true}, \text{false}\}) \\[10pt] \frac{\Gamma(x) = \tau \text{ or } \text{fields}(C)(x) = \tau}{\Gamma \vdash x : \tau} \text{ (T-VAR)} \quad \frac{\text{inside a method of class } C}{\Gamma \vdash \text{this} : C} \text{ (T-THIS)} \end{array}$$

(T-VAR is one rule with two side conditions in the code, because a bare identifier is either a local/parameter lookup in Γ or an implicit field read through $\text{fields}(C)$; the Typing tab’s note line reports which one fired, e.g. “ $\Gamma(v) = \text{int}$ ” or “ $\text{fields}(\text{Cell})(f) = \text{int}$ ”. T-THIS is undefined — reported as an error, typed $?$ — inside **main**, which has no receiver.)

Object and array construction.

$$\frac{C \text{ is declared, } C \neq \text{main class}}{\Gamma \vdash \text{new } C() : C} \quad (\text{T-NEW}) \qquad \frac{\Gamma \vdash e : \text{int}}{\Gamma \vdash \text{new int}[e] : \text{int}[]} \quad (\text{T-NEWARR})$$

Operators.

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \oplus e_2 : \text{int}} \quad (\text{T-ARITH}) \quad (\oplus \in \{+, -, *, /\})$$

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \bowtie e_2 : \text{boolean}} \quad (\text{T-CMP}) \quad (\bowtie \in \{<, \leq, >, \geq\})$$

$$\frac{\Gamma \vdash e_1 : \text{boolean} \quad \Gamma \vdash e_2 : \text{boolean}}{\Gamma \vdash e_1 \&\& e_2 : \text{boolean}} \quad (\text{T-AND}) \qquad \frac{\Gamma \vdash e_1 : \text{boolean} \quad \Gamma \vdash e_2 : \text{boolean}}{\Gamma \vdash e_1 || e_2 : \text{boolean}} \quad (\text{T-OR})$$

$$\frac{\Gamma \vdash e : \text{boolean}}{\Gamma \vdash !e : \text{boolean}} \quad (\text{T-NOT})$$

(Parenthesization, `mj_expr_parens`, is typing-*transparent*: the checker returns the child's own `Derivation` unchanged, so no T-PARENS node ever appears in a derivation tree.)

Arrays.

$$\frac{\Gamma \vdash e_1 : \text{int}[] \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1[e_2] : \text{int}} \quad (\text{T-LOOKUP}) \qquad \frac{\Gamma \vdash e : \text{int}[]}{\Gamma \vdash e.\text{length} : \text{int}} \quad (\text{T-LENGTH})$$

Strings (extension over the base BNF).

$$\frac{\Gamma \vdash e_1 : \text{String} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1.\text{charAt}(e_2) : \text{String}} \quad (\text{T-CHARAT})$$

$$\frac{\Gamma \vdash e_1 : \text{String} \quad \Gamma \vdash e_2 : \text{String}}{\Gamma \vdash e_1.\text{concat}(e_2) : \text{String}} \quad (\text{T-CONCAT}) \qquad \frac{\Gamma \vdash e : \text{String}}{\Gamma \vdash e.\text{length}() : \text{int}} \quad (\text{T-STRLEN})$$

(T-CHARAT returns a *one-character* `String`, not a `char` — the language has no character type, matching the design note's "strings, no chars" decision.)

Method invocation.

$$\frac{\Gamma \vdash e_0 : C \quad \text{mtype}(m, C) = (\tau_1, \dots, \tau_n) \rightarrow \tau \quad \Gamma \vdash e_i : \sigma_i \quad \sigma_i \rightsquigarrow \tau_i \quad (1 \leq i \leq n)}{\Gamma \vdash e_0.m(e_1, \dots, e_n) : \tau} \quad (\text{T-INVK})$$

This is, rule-for-rule, Featherweight Java's T-INVK [2, Fig. 1], adapted from FJ's purely functional core to MiniJava's imperative one (arguments are still evaluated for their values, not for effects that matter to typing). The Typing tab's note line for this rule literally prints the `mtype` judgment, e.g. "`mtype(set, Cell) = (int) -> int`".

2.5 Statement well-formedness rules (WF-*)

Judgment: $\Gamma \vdash s \text{ ok}$ — “under Γ , statement s is well-formed.” Statements have no type of their own (MiniJava’s imperative core is outside Featherweight Java’s purely functional scope), so their judgment concludes `ok` rather than a type — the standard move for adding an imperative statement layer on top of an expression type system (cf. Pierce [1, Ch. 13] on typing references and commands).

$$\begin{array}{c}
\frac{\Gamma \vdash s_1 \text{ ok} \cdots \Gamma \vdash s_k \text{ ok}}{\Gamma \vdash \{s_1; \dots; s_k\} \text{ ok}} \text{ (WF-BLOCK)} \\
\\
\frac{\Gamma \vdash e : \text{boolean} \quad \Gamma \vdash s_1 \text{ ok (then)} \quad \Gamma \vdash s_2 \text{ ok (else)}}{\Gamma \vdash \text{if}(e) s_1 \text{ else } s_2 \text{ ok}} \text{ (WF-IF)} \\
\\
\frac{\Gamma \vdash e : \text{boolean} \quad \Gamma \vdash s \text{ ok}}{\Gamma \vdash \text{while}(e) s \text{ ok}} \text{ (WF-WHILE)} \\
\\
\frac{\Gamma \vdash e : \text{int} \text{ or } \Gamma \vdash e : \text{String}}{\Gamma \vdash \text{System.out.println}(e) \text{ ok}} \text{ (WF-PRINT)} \\
\\
\frac{\Gamma(x) = \tau \text{ or } \text{fields}(C)(x) = \tau \quad \Gamma \vdash e : \sigma \quad \sigma \rightsquigarrow \tau}{\Gamma \vdash x = e \text{ ok}} \text{ (WF-ASSIGN)} \\
\\
\frac{\Gamma(a) = \text{int}[] \text{ or } \text{fields}(C)(a) = \text{int}[] \quad \Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash a[e_1] = e_2 \text{ ok}} \text{ (WF-ARRASSIGN)}
\end{array}$$

An undeclared variable does not abort the derivation: WF-ASSIGN and T-VAR still produce a node (typed `?`, or with a “ $x \notin \Gamma$ ” note) so the rest of the program keeps checking — the same Hazel-style “hole, don’t cascade” policy as §2.3, applied at the statement level.

2.6 Method well-formedness: M-OK

$$\frac{\Gamma = \{\text{this} : C, x_1 : \tau_1, \dots\} \quad \Gamma \vdash s_1 \text{ ok} \cdots \Gamma \vdash s_k \text{ ok} \quad \Gamma \vdash e_{\text{ret}} : \sigma \quad \sigma \rightsquigarrow \tau_{\text{ret}}}{\text{M-OK} \vdash \text{method } C.m \text{ ok}} \text{ (M-OK)}$$

is the derivation *root* the Typing tab renders per method (and once for `main`, with no `return` premise and `this` omitted from Γ). The name and shape mirror Featherweight Java’s method judgment $M \text{ OK IN } C$ [2, Fig. 1], again re-purposed for an imperative statement sequence instead of FJ’s single return expression.

2.7 Method overriding

Outside the per-expression/statement rules, the checker enforces one more well-formedness condition on the class table: an overriding method must have *identical* parameter types and a *covariant* return type,

$$\frac{C <: D \quad \text{mtype}(m, D) = (\tau_1, \dots, \tau_n) \rightarrow \tau \quad \text{mtype}(m, C) = (\tau_1, \dots, \tau_n) \rightarrow \tau' \quad \tau' <: \tau}{\text{override of } m \text{ in } C \text{ is well-formed}} \text{ (OVERRIDE)}$$

which is FJ’s override condition [2, §2.3] generalized from FJ’s invariant-everything rule to Java’s actual covariant-return rule (JLS §8.4.8.3, since Java 5) — MiniJava has no overloading, so “the” overridden signature is unambiguous.

2.8 Theoretical sources

- **Natural-deduction typing judgments, rule-naming convention, and the T-* prefix** follow Pierce’s *Types and Programming Languages* [1], the standard textbook presentation of $\Gamma \vdash e : \tau$ typing rules as premises-over-a-bar proof trees — exactly the shape the Typing tab renders.
- **The object layer** — mtype, fields, T-INVK, T-NEW, nominal subtyping seeded from **extends**, and the *M* OK method judgment — is Featherweight Java [2], the standard minimal core calculus for a Java-like nominally-typed OO language. FJ’s auxiliary functions and two of its rule names are used unchanged; MiniJava is close enough to FJ’s object model (fields, single inheritance, method invocation, **this**, **new**) that no adaptation was needed there.
- **The imperative statement layer** (WF-*) has no FJ analogue — FJ is purely functional, with no assignment, loops, or a store. The $\Gamma \vdash s \text{ ok}$ judgment form and its composition with the expression layer follow the generic treatment of imperative extensions to a typed calculus in Pierce [1, Ch. 13].
- **Typed holes and hole consistency** (§2.3) follow Hazel’s live, always-typed editor calculus [3, 4], itself built on the consistency relation from gradual typing [5].

3 Operational Semantics

Block-MiniJava implements two independent evaluators over the same block AST, deliberately keeping them structurally different so that agreement between them is evidence, not a tautology (design note §6–§8; [research-notes/abstract-machine-analysis.md](#)):

	Vehicle 1 — abstract machine	Vehicle 2 — substitution
file	<code>minijavaMachine.ts</code>	<code>minijavaSubstitution.ts</code>
scope	the full language (Models A and B)	the pure fragment only
state	control + frame stack + heap	one block tree
step	transition rule on a configuration	literal rewrite of a redex
tab	CESK, A vs B	Rewrite

3.1 Vehicle 1: a store-bearing abstract machine, parameterized by value model

3.1.1 Configurations

A machine state is a tuple

$$\Sigma = \langle C, \varphi:\Phi, H \rangle$$

where *C* is the *control* (a statement, an expression, a produced *value*, or *done*); $\varphi:\Phi$ is a non-empty *stack of activation frames*, top frame φ , the rest Φ (pushed on method call, popped on return); and $H : \text{Loc} \rightarrow \text{HeapObj}$ is the *store*. Each frame is itself a small CEK-style configuration,

$$\varphi = \langle \text{self}, \rho, \kappa \rangle, \quad \rho : \text{Var} \rightarrow \text{Value}, \quad \kappa : \text{Kont}^*$$

with ρ the frame’s local environment (parameters and locals, held *by copy*) and κ its pending-work stack, innermost first. This is a **CESK machine** (Control, Environment, Store, Continuation) in the sense of Felleisen and Friedman’s CEK machine [6] extended with an explicit, imperative-language store *S/H* as formalized by Might and Van Horn’s abstracting-abstract-machines treatment of CESK machines [7], further extended with an explicit *stack of frames* (rather than one shared continuation) to support method call/return — the *dump* of Landin’s

original SECD machine [8]. The project’s own research note classifies this precisely as “a CESK-style machine with an essential mutable store, over an explicit call stack of activation frames” (`research-notes/abstract-machine-analysis.md`).

Values diverge by model.

$$v ::= n \mid b \mid s \mid \text{null} \mid \underbrace{\text{Ref}(\ell)}_{\text{Model A only}} \mid \underbrace{\text{Obj}(C, \overline{f=v}) \mid \text{Arr}(\overline{v})}_{\text{Model B only}}$$

Under **Model A**, $\text{Ref}(\ell)$ is the *only* way to reach an object or array; $H(\ell)$ holds the actual Obj/Arr , so aliasing is real and field write is a store mutation. Under **Model B**, $H = \emptyset$ always (the “no-store invariant”): objects and arrays *are* values, copied by binding, and field/array update rebuilds a fresh value rather than mutating anything. The two models share every other part of the machine (control flow, continuations, frame-stack discipline), so **step** is one function parameterized by $\text{model} \in \{A, B\}$, and running both models on the same program is a genuine lockstep execution, not two separate interpreters that merely happen to agree.

3.1.2 Continuations

group	continuation frames (κ)
statements	KStmtSeq, KLoop, KIf, KWhile, KPrint, KAssign, KArrAssignIdx, KArrAssignVal
operators	KNot, KAnd, KOr, KBinL $\rightarrow$ KBinR
arrays/strings	KLookupArr $\rightarrow$ KLookupIdx, KLength, KNewArr, KCharAtStr $\rightarrow$ KCharAtIdx, KStrLength
method call	KCallRecv $\rightarrow$ KCallArgs (receiver, then arguments left-to-right, then push a frame)

3.1.3 Transition rules

Rules are grouped by category; each is named exactly as `lastRule` records it. A step $\Sigma \rightarrow_r \Sigma'$ reads “ Σ steps to Σ' by rule r .” Rules marked *salient* correspond 1:1 to a rewrite of the substitution semantics (§3.3); rules marked *administrative* are bookkeeping the substitution semantics never names (entering a block, dispatching `if/while`, pushing an operand continuation, ...).

Literals and variables (administrative).

$$\begin{array}{c}
\frac{}{\langle \text{Expr}(\ell), \varphi, \Phi, H \rangle \rightarrow_{\text{LIT}} \langle \text{Value}(v(\ell)), \varphi, \Phi, H \rangle} \text{ (LIT)} \\
\\
\frac{x \in \text{dom}(\rho)}{\langle \text{Expr}(x), \langle \text{self}, \rho, \kappa \rangle, \Phi, H \rangle \rightarrow_{\text{VAR}} \langle \text{Value}(\rho(x)), \langle \text{self}, \rho, \kappa \rangle, \Phi, H \rangle} \text{ (VAR)} \\
\\
\frac{x \notin \text{dom}(\rho) \quad H(\text{self}).\text{fields}(x) = v \text{ (A)}, \text{ or } \text{self}.\text{fields}(x) = v \text{ (B)}}{\langle \text{Expr}(x), \varphi, \Phi, H \rangle \rightarrow_{\text{FIELD-READ}} \langle \text{Value}(v), \varphi, \Phi, H \rangle} \text{ (FIELD-READ)} \\
\\
\frac{}{\langle \text{Expr}(\text{this}), \langle \text{self}, \rho, \kappa \rangle, \Phi, H \rangle \rightarrow_{\text{THIS}} \langle \text{Value}(\underbrace{\text{Ref}(\text{self})}_A \text{ or } \underbrace{\text{self}}_B), \varphi, \Phi, H \rangle} \text{ (THIS)}
\end{array}$$

Control flow (administrative except where noted).

$$\frac{}{\langle \text{Done}, \langle \dots, \text{KStmtSeq}(s_{\text{next}}) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{SEQ}} \begin{cases} \langle \text{Stmt}(s_{\text{next}}), \langle \dots, \text{KStmtSeq}(s'_{\text{next}}) : \kappa \rangle : \Phi, H \rangle & s_{\text{next}} \neq \perp \\ \langle \text{Done}, \langle \dots, \kappa \rangle : \Phi, H \rangle & s_{\text{next}} = \perp \end{cases} \quad (\text{SEQ} / \text{SEQ-END})}$$

$$\frac{b \in \{\text{true}, \text{false}\}}{\langle \text{Value}(b), \langle \dots, \text{KIf}(s_{\text{then}}, s_{\text{else}}) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{IF-TRUE/FALSE}} \langle \text{enter}(b ? s_{\text{then}} : s_{\text{else}}), \dots \rangle} \quad (\text{IF-TRUE} / \text{IF-FALSE})$$

$$\frac{}{\langle \text{Value}(b), \langle \dots, \text{KWhile}(e, s) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{WHILE-ENTER/EXIT}} \begin{cases} \text{enter}(s) \text{ with } \text{KLoop}(e, s) : \kappa & b = \text{true} \\ \text{Done with } \kappa & b = \text{false} \end{cases} \quad (\text{WHILE-ENTER} / \text{WHILE-EXIT} / \text{LOOP})}$$

(LOOP re-focuses the **while** statement once its body's KLoop continuation is reached, restarting the condition.)

$$\frac{}{\langle \text{Value}(v), \langle \dots, \text{KAssign}(x) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{ASSIGN}} \langle \text{Done}, \langle \dots, \rho[x \mapsto v], \kappa \rangle : \Phi, H \rangle} \quad (\text{ASSIGN})$$

$$\frac{}{\langle \text{Value}(v), \langle \text{self} = \ell, \dots, \text{KAssign}(f) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{FIELD-WRITE}} \langle \text{Done}, \varphi, \Phi, H[\ell.f \mapsto v] \rangle} \quad (\text{FIELD-WRITE})$$

(Model A: the store mutates.)

$$\frac{}{\langle \text{Value}(v), \langle \text{self} = o, \dots, \text{KAssign}(f) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{FIELD-UPDATE}} \langle \text{Done}, \langle o[f \mapsto v], \dots \rangle, \Phi, H \rangle} \quad (\text{FIELD-UPDATE})$$

(Model B: a fresh object replaces o .) The pair FIELD-WRITE/FIELD-UPDATE is where Models A and B provably diverge: the built-in *Aliasing Contrast* example prints 4141 under A (a shared heap cell observes both writes) and 41 under B (each alias holds an independent copy) after the same 51 steps on both machines — the machine-level demonstration of the design note's core distinction (§2–§4).

$$\frac{}{\langle \text{Value}(v), \langle \dots, \text{KPrint} : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{PRINTLN}} \langle \text{Done}, \varphi, \Phi, H \rangle, \text{output} += \text{show}(v)} \quad (\text{PRINTLN})$$

$$\frac{}{\langle \text{Value}(v), \langle \dots, \text{KArrAssignVal}(a, i) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{ARR-WRITE/ARR-UPDATE}} \dots} \quad (\text{ARRAY-WRITE} / \text{ARRAY-UPDATE})$$

(again bifurcating exactly as field write: Model A mutates $H(\ell)$ in place (ARRAY-WRITE); Model B rebinds a to a new array value with index i replaced (ARRAY-UPDATE).)

Operators (mixed). Left-to-right, call-by-value evaluation is enforced by the KBinL $\rightarrow$ KBinR continuation pair (administrative: BINOP, BINOP-RIGHT); the salient fold rules are

ADD, SUB, MUL, DIV, LESS, LEQ, GT, GEQ, CONCAT

e.g.

$$\frac{}{\langle \text{Value}(n_2), \langle \dots, \text{KBinR}(\text{add}, n_1) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{ADD}} \langle \text{Value}(n_1 + n_2), \varphi, \Phi, H \rangle} \quad (\text{ADD})$$

$$\frac{n_2 \neq 0}{\langle \text{Value}(n_2), \langle \dots, \text{KBinR}(\text{div}, n_1) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{DIV}} \langle \text{Value}(\text{trunc}(n_1/n_2)), \varphi, \Phi, H \rangle} \quad (\text{DIV})$$

(division truncates toward zero and gets *stuck* on $n_2 = 0$, matching Java's `ArithmeticException` rather than silently producing a value). Boolean connectives short-circuit via dedicated continuations:

$$\frac{}{\langle \text{Value}(\text{false}), \langle \dots, \text{KAnd}(e_2) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{AND-SC}} \langle \text{Value}(\text{false}), \varphi, \Phi, H \rangle} \quad (\text{AND-SHORT-CIRCUIT})$$

$$\frac{}{\langle \text{Value}(\text{true}), \langle \dots, \text{KOr}(e_2) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{OR-SC}} \langle \text{Value}(\text{true}), \varphi, \Phi, H \rangle} \quad (\text{OR-SHORT-CIRCUIT})$$

with AND/OR firing once the (necessarily evaluated) right operand comes back, and NOT negating a boolean directly.

Strings.

$$\frac{0 \leq i < |s|}{\langle \text{Value}(i), \langle \dots, \text{KCharAtIdx}(s) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{CHAR-AT}} \langle \text{Value}(s[i]), \varphi, \Phi, H \rangle} \quad (\text{CHAR-AT})$$

$$\frac{}{\dots \longrightarrow_{\text{STR-LENGTH}} \langle \text{Value}(|s|), \dots \rangle} \quad (\text{STR-LENGTH})$$

(i out of $[0, |s|)$ is *stuck*: a string index out of bounds, Java's `StringIndexOutOfBoundsException`.)

Arrays.

$$\frac{n \geq 0}{\langle \text{Value}(n), \langle \dots, \text{KNewArr} : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{NEW-ARRAY}} \begin{cases} \langle \text{Value}(\text{Ref}(\ell)), \varphi, \Phi, H[\ell \mapsto \bar{0}^n] \rangle & \text{Model A} \\ \langle \text{Value}(\text{Arr}(\bar{0}^n)), \varphi, \Phi, H \rangle & \text{Model B} \end{cases}} \quad (\text{NEW-ARRAY})$$

$$\frac{0 \leq i < |arr|}{\dots \longrightarrow_{\text{ARRAY-READ}} \langle \text{Value}(arr[i]), \dots \rangle} \quad (\text{ARRAY-READ}) \quad \frac{}{\dots \longrightarrow_{\text{ARRAY-LENGTH}} \langle \text{Value}(|arr|), \dots \rangle} \quad (\text{ARRAY-LEN})$$

Objects and method call/return.

$$\frac{C \text{ is declared, } C \neq \text{main class}}{\langle \text{Expr}(\text{new } C()), \varphi, \Phi, H \rangle \longrightarrow_{\text{NEW}} \begin{cases} \langle \text{Value}(\text{Ref}(\ell)), \varphi, \Phi, H[\ell \mapsto \text{Obj}(C, \bar{f}=\bar{0})] \rangle & \text{A} \\ \langle \text{Value}(\text{Obj}(C, \bar{f}=\bar{0})), \varphi, \Phi, H \rangle & \text{B} \end{cases}} \quad (\text{NEW})$$

$$\frac{v_0 \text{ resolves to an object of dynamic class } C \quad \text{mtype}(m, C) = (\tau_1, \dots, \tau_n) \rightarrow \tau \quad |args| = n}{\langle \text{Value}(v_n), \langle \dots, \text{KCallArgs}(v_0, \bar{v}) : \kappa \rangle : \Phi, H \rangle \longrightarrow_{\text{CALL}} \langle \text{Stmt}(\text{body}(m, C)), \varphi_{\text{new}} : \varphi : \Phi, H \rangle} \quad (\text{CALL})$$

where $\varphi_{\text{new}} = \langle \text{self} = v_0, [x_i \mapsto v_i]_{i=1}^n, [] \rangle$ is a fresh frame — dynamic dispatch: the method looked up by mtype is resolved from v_0 's *dynamic* class, exactly as in FJ/Java, not from the receiver expression's static type.

$$\frac{\kappa = [], \text{frame has a return expression not yet focused}}{\langle \text{Done}, \varphi, \Phi, H \rangle \longrightarrow_{\text{RETURN-FOCUS}} \langle \text{Expr}(e_{\text{ret}}), \varphi, \Phi, H \rangle} \quad (\text{RETURN-FOCUS})$$

$$\frac{}{\langle \text{Value}(v), \varphi : \Phi, H \rangle \longrightarrow_{\text{RETURN}} \langle \text{Value}(v), \Phi, H \rangle} \quad (\text{RETURN}) \quad \frac{|\Phi| = 0, C = \text{Done}}{\Sigma \longrightarrow_{\text{HALT}} \text{status} := \text{done}} \quad (\text{HALT})$$

3.2 Vehicle 2: substitution-based reduction semantics (pure fragment)

The Rewrite tab implements a classical *small-step reduction semantics*: every state is a literal syntax tree, and a step contracts one *redex* in place, following Plotkin’s structural operational semantics [9] as refined into the *reduction-semantics-with-evaluation-contexts* style of Felleisen and Hieb [10], the style taught and tooled in *Semantics Engineering with PLT Redex* [11] — the design note’s own vocabulary (“redex,” “salient reduction,” congruence-by-recursive-descent) matches that tradition directly.

3.2.1 Scope: the pure fragment

Vehicle 2 covers only programs whose every method body is a straight-line sequence of simple assignments followed by a `return` (`analyzeMethod` in `minijavaSubstitution.ts`); `if`, `while`, `println`, and array writes make a method *impure*, and the stepper reports *why* rather than silently changing semantics. `main` itself is restricted to exactly one `System.out.println(...)`. This mirrors the classic move of carving out a substitution-only calculus (the λ -calculus core, or FJ’s purely functional method bodies) from a full imperative language, so that pure substitution — no environment, no store — is even meaningful.

3.2.2 Values and evaluation contexts

$$v ::= n \mid b \mid s \mid C\{f_1=v_1, \dots, f_k=v_k\}$$

Congruence (“where does the next redex live”) is implemented as left-to-right, outermost-in recursive descent that stops at the first non-value child — an *implicit* evaluation-context grammar

$$E ::= [] \mid E \oplus e \mid v \oplus E \mid \dots \mid E.m(\bar{e}) \mid v.m(\bar{v}, E, \bar{e})$$

(one production per operator/constructor, all call-by-value, left argument first) that `stepNode`’s recursive structure encodes directly rather than naming as data — the standard *reduction-free* implementation strategy for evaluation contexts [10].

3.2.3 Reduction rules

Fold rules (identical arithmetic/relational/string semantics to Vehicle 1, by construction — both call the same truncating-division and 1-character-`charAt` conventions):

$$\begin{array}{c}
\frac{}{n_1 \oplus n_2 \longrightarrow n_1 \oplus_{\mathbb{Z}} n_2} \text{ (ADD/SUB/MUL/DIV)} \qquad \frac{}{n_1 \bowtie n_2 \longrightarrow n_1 \bowtie_{\mathbb{Z}} n_2} \text{ (LESS/LEQ/GT/GEQ)} \\
\\
\frac{}{\text{false} \ \&\& \ e_2 \longrightarrow \text{false}} \text{ (AND-SHORT-CIRCUIT)} \qquad \frac{}{\text{true} \ \&\& \ b \longrightarrow b} \text{ (AND)} \\
\\
\frac{}{\text{true} \ || \ e_2 \longrightarrow \text{true}} \text{ (OR-SHORT-CIRCUIT)} \qquad \frac{}{\text{false} \ || \ b \longrightarrow b} \text{ (OR)} \\
\\
\frac{}{!b \longrightarrow \neg b} \text{ (NOT)} \qquad \frac{}{(v) \longrightarrow v} \text{ (PARENS)} \qquad \frac{}{"s_1".concat("s_2") \longrightarrow "s_1s_2"} \text{ (CONCAT)} \\
\\
\frac{0 \leq i < |s|}{"s".charAt(i) \longrightarrow "s[i]} \text{ (CHAR-AT)} \qquad \frac{}{"s".length() \longrightarrow |s|} \text{ (STR-LENGTH)} \\
\\
\frac{C \text{ declared, } C \neq \text{main class}}{\text{new } C() \longrightarrow C\{f_1=\text{default}(\tau_1), \dots, f_k=\text{default}(\tau_k)\}} \text{ (NEW)}
\end{array}$$

3.2.4 The β rule: method call

Method call is this semantics' one true β -rule, and its side conditions are the most intricate part of the implementation:

$$\begin{array}{l}
v_0 = C\{\overline{f=u}\}, \quad \text{mdef}(m, C) = (x_1, \dots, x_n)\{y_1:=e_1; \dots; y_k:=e_k; \text{ return } e_{\text{ret}}\} \\
\text{mdef}(m, C) \text{ pure (assignments only)}, \quad n = |v_1, \dots, v_n| \\
\rho_0 = [x_i \mapsto v_i]_{i=1}^n, \quad \phi_0 = [f \mapsto u] \\
\forall j \in 1..k : w_j = \sigma(e_j; \rho_{j-1}, \phi_{j-1}) \text{ is itself a value}, \quad (\rho_j, \phi_j) = \text{update}(\rho_{j-1}, \phi_{j-1}, y_j \mapsto w_j) \\
\hline
v_0.m(v_1, \dots, v_n) \longrightarrow \sigma(e_{\text{ret}}; \rho_k, \phi_k)
\end{array} \tag{CALL}$$

where $\sigma(e; \rho, \phi)$ substitutes each free identifier x by $\text{copyFresh}(\rho(x))$ (or $\phi(x)$ if x is a field, i.e. an implicit **this.x**) and each occurrence of **this** by the *freshly rebuilt* object $C\{f \mapsto \phi(f)\}$ — so **this** inside assignment $j+1$ already reflects assignment j 's functional update, matching Model B's semantics exactly. **Every substituted occurrence gets a fresh id** (**copyFresh**); this is the *structure-preservation invariant* — no two parents ever share a child node, i.e. substitution here is the classical *capture-avoiding, sharing-free* substitution $e[v/x]$ of the λ -calculus, not the graph-sharing substitution that would secretly reintroduce Model A's aliasing.

The requirement that each w_j *already be a value* after substitution (checked by **isValueNode**, else the method is reported outside the pure fragment) is a deliberate restriction to *non-duplicating*, call-by-value β : because σ copies $\rho(x)$ verbatim into every occurrence of x , allowing w_j to be an unevaluated redex would risk re-doing the same computation at every occurrence — the classical call-by-name/call-by-value duplication hazard that β_v -reduction (call-by-value β , e.g. Plotkin [12]) exists to avoid.

3.3 Operational correspondence between the two vehicles

The Rewrite tab's companion check (implemented in **test/run_subst.js** and surfaced live in the UI) replays the same program on the Model B machine and asserts that the two traces agree on every *salient* rule and on the final value, where salient is exactly the set

`{ADD, SUB, MUL, DIV, LESS, LEQ, GT, GEQ, NOT, AND, AND-SHORT-CIRCUIT, OR, OR-SHORT-CIRCUIT, NEW, CALL, CONCAT, CHAR-AT, STR-LENGTH}`

Every other machine rule (**seq**, **if-***, **while-***, **var**, **field-read**, **push/pop-frame**, the **K*** continuation dispatches, ...) is *administrative*: it exists because the machine represents control and environment explicitly, where the substitution semantics simply rewrites the tree and looks nothing up. The claim being tested — that “environment lookup on the machine is lazy substitution” (README, *Semantics & steppers*) — is precisely the informal statement proved formally, for general reduction semantics/abstract-machine pairs, by Danvy's *syntactic correspondence* technique (deriving an abstract machine from a reduction semantics by refocusing, and vice versa) [13, 14], and is the operational half of the progress/preservation methodology that soundness proofs use to relate a reduction relation to a machine [15]. Here the correspondence is checked *empirically*, by exhaustive comparison of salient-rule traces on a fixed program suite (**test/run_subst.js**, 35 programs at the time of writing) plus a 400-program differential fuzz suite that additionally cross-checks both machine models, the substitution stepper, and a legacy evaluator against each other (**test/run_fuzz.js**) — it is not machine-checked in a proof assistant, unlike the sibling Live-Types project's Coq-mechanized metatheory.

3.4 An auxiliary evaluation rule system: mark-and-sweep garbage collection

Model A's heap accumulates unreachable objects as a normal side effect of 'new'/field-write; **src/core/semantics/gc.ts** implements a classical two-phase *tracing* collector over that heap:

1. **Mark** — compute $\text{reachable}(H) \subseteq \text{dom}(H)$ as the least fixed point of a root set (every Ref reachable from a live frame’s ρ , **self**, or κ) closed under following object fields and array elements.
2. **Sweep** — restrict H to $\text{reachable}(H)$, dropping every other binding.

This is the original mark-and-sweep algorithm as introduced for Lisp by McCarthy [16] and given its modern, general treatment (root sets, tracing vs. reference counting, generational variants) in Jones, Hosking, and Moss’s *The Garbage Collection Handbook* [17]. It is deliberately *not* part of the machine’s core transition relation $\longrightarrow$: GC is triggered externally (a UI action or an explicit test call to `runGC`), observably shrinks H , and is proved — by the GC test suite (`test/run_gc.js`, 14 cases) — to never change a program’s observable output, i.e. it is a *sound* store transformation: $\forall \Sigma. \text{run}(\Sigma) = \text{run}(\text{gc}(\Sigma))$ on output, even though the two runs’ heaps differ.

3.5 Theoretical sources

- **The CESK abstract machine** follows Felleisen and Friedman’s CEK machine [6], extended with an explicit store as formalized in Might and Van Horn’s CESK presentation [7]; the explicit call-stack-of-frames (rather than one shared continuation) traces back to the *dump* component of Landin’s SECD machine [8], the historical origin of explicit function-call/return machinery in an abstract machine.
- **The substitution/reduction semantics** follows Plotkin’s structural operational semantics [9] in the evaluation-contexts style of Felleisen and Hieb [10], as taught and toolled by *PLT Redex* [11]; call-by-value β -reduction specifically follows Plotkin’s call-by-value λ -calculus [12].
- **The operational-correspondence check** between the two vehicles instantiates, empirically rather than by proof, the syntactic-correspondence methodology relating reduction semantics to abstract machines [13, 14], which is also the technique underlying Wright and Felleisen’s syntactic type-soundness proofs [15].
- **Mark-and-sweep garbage collection** follows McCarthy’s original algorithm [16], per the modern treatment in [17].

4 Summary: rule families and their theoretical lineage

Rule family	Where	Theoretical source
$\Gamma \vdash e : \tau, \text{T-}^*$	<code>typeChecker.ts</code> , expression rules	Pierce, <i>TAPL</i> [1] (judgment form and rule-naming convention)
T-INVK, T-NEW, mtype, fields, M-OK, OVERRIDE	<code>typeChecker.ts/classTable.ts</code>	Featherweight Java [2] (names, auxiliary functions, and rule shapes reused directly)
WF-*, $\Gamma \vdash s \text{ ok}$	<code>typeChecker.ts</code> , statement rules	Imperative-extension treatment of a typed calculus [1, Ch. 13]; no FJ analogue (FJ has no statements)
?, $\rightsquigarrow$ (hole consistency)	<code>ty.ts</code> , <code>classTable.ts</code>	Hazel’s typed holes [3, 4], built on gradual-typing consistency [5]
CESK machine, $\Sigma \longrightarrow \Sigma'$	<code>minijavaMachine.ts</code>	Felleisen–Friedman CEK [6]; CESK per Might–Van Horn [7]; frame stack per Landin’s SECD dump [8]

Rule family	Where	Theoretical source
Substitution rewrite, $e \longrightarrow e'$	<code>minijavaSubstitution.ts</code>	Plotkin’s SOS [9]; Felleisen–Hieb evaluation contexts [10]; PLT Redex style [11]
CALL (β- reduction for method invocation)	<code>minijavaSubstitution.ts</code> , <code>beta()</code>	Call-by-value β_v [12]
Operational correspondence (salient-rule agreement)	<code>test/run_subst.js</code>	Syntactic correspondence between reduction semantics and abstract machines [13, 14]; Wright–Felleisen soundness methodology [15]
Mark-and- sweep GC	<code>gc.ts</code>	McCarthy [16]; modern treatment in [17]

Note on scope

This note reconstructs the rules *as implemented*; it does not attempt a soundness proof (progress/preservation) relating §2 to §3, and the operational-correspondence claim of §3.3 is currently checked by exhaustive testing, not mechanized. Both are natural next artifacts if this project follows the sibling Live-Types work toward a machine-checked metatheory.

References

- [1] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [2] A. Igarashi, B. C. Pierce, and P. Wadler. Featherweight Java: a minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 23(3):396–450, 2001. (Originally OOPSLA 1999.)
- [3] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer. Hazelnut: a bidirectionally typed structure editor calculus. In *POPL*, 2017.
- [4] C. Omar, D. Moon, A. Blinn, I. Voysey, N. Collins, and R. Chugh. Live functional programming with typed holes. In *POPL*, 2019.
- [5] J. G. Siek and W. Taha. Gradual typing for functional languages. In *Scheme and Functional Programming Workshop*, 2006.
- [6] M. Felleisen and D. P. Friedman. Control operators, the SECD machine, and the λ -calculus. In *Formal Description of Programming Concepts III*, 1986.
- [7] M. Might and D. Van Horn. Abstracting abstract machines. In *ICFP*, 2010.
- [8] P. J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.
- [9] G. D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Aarhus University, 1981.
- [10] M. Felleisen and R. Hieb. The revised report on the syntactic theories of sequential control and state. *Theoretical Computer Science*, 103(2):235–271, 1992.

- [11] M. Felleisen, R. B. Findler, and M. Flatt. *Semantics Engineering with PLT Redex*. MIT Press, 2009.
- [12] G. D. Plotkin. Call-by-name, call-by-value and the λ -calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.
- [13] O. Danvy. A rational deconstruction of Landin’s SECD machine. In *IFL*, 2004.
- [14] M. S. Ager, D. Biernacki, O. Danvy, and J. Midtgaard. A functional correspondence between evaluators and abstract machines. In *PPDP*, 2003.
- [15] A. K. Wright and M. Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.
- [16] J. McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4):184–195, 1960.
- [17] R. Jones, A. Hosking, and E. Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2011.